University of Scholars

Bazar-Trace: Offline-First PWA Stock & Inventory Management System

Built with HTML, CSS, Vanilla JS, Node.js (Custom HTTP Server) & Oracle Database (Local)

> Presenter_Metadata
Name: Arif Hossen
ID: 231010687
Dept: Computer Science & Engineering (CSE)

> Supervision_Metadata
Supervisor: Reduanul Bari Subhon sir
Mentor: Israt Jahan miss

What is Bazar-Trace?

Vanilla SPA

Built purely with raw ES Modules, HTML5, and CSS variables—zero heavy frameworks.

Offline-First Sync Engine

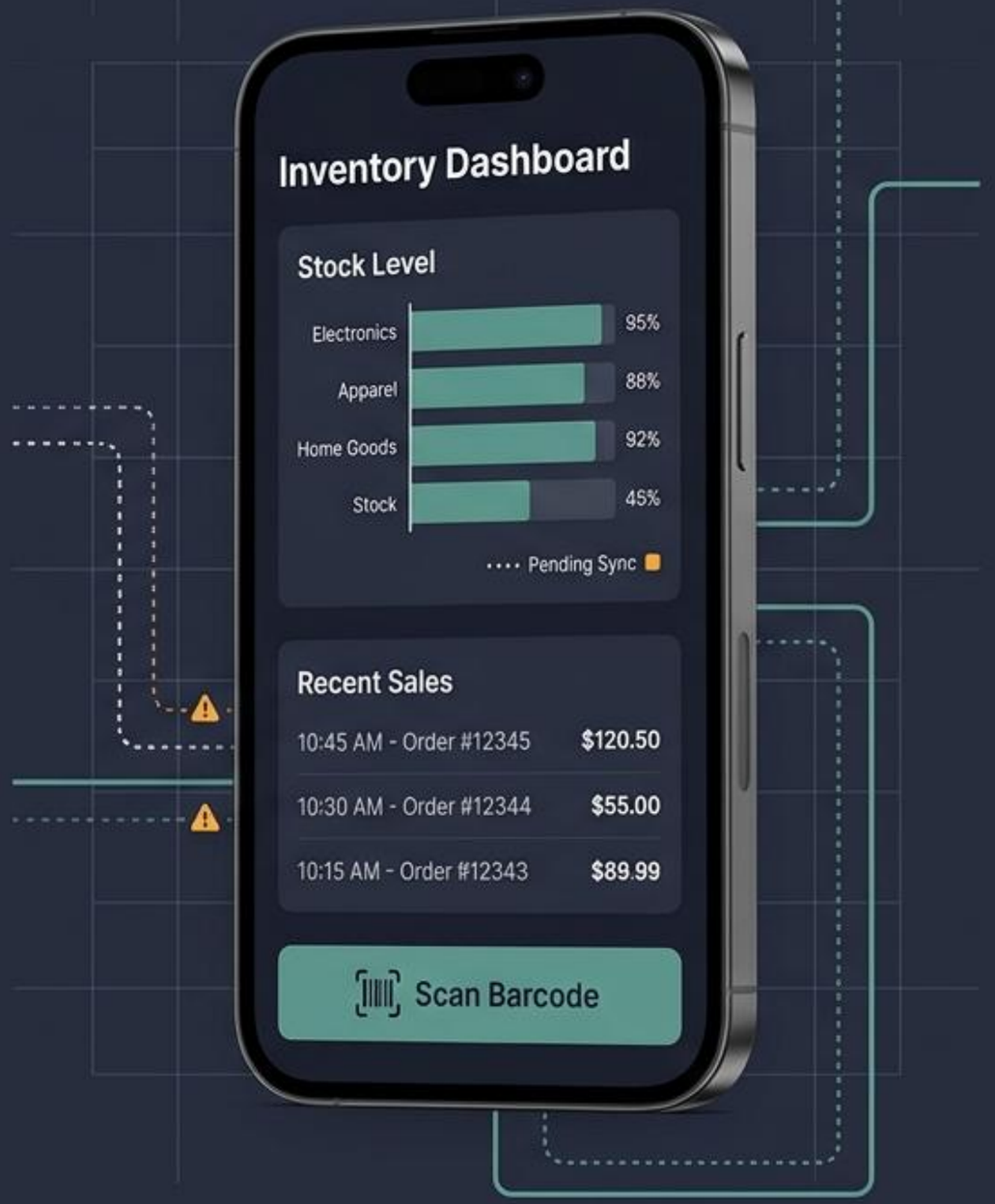
Records transactions and looks up local product data even during complete network drops.

Role-Based Security

Strict functional separation between admin management interfaces and staff operational views.

Mobile-First Design

Hyper-optimized interface for low-end retail devices and simple mobile browsers.



The Catalyst: Why Cloud ERPs Fail Local Retail

	Legacy Cloud ERPs	Bazar-Trace Local PWA
Network Reliance	✕ Fails during Bangladesh's frequent internet instability.	✓ 100% operational offline via Service Workers.
Operational Cost	✕ Requires expensive desktop hardware and heavy recurring software licenses.	✓ Runs on existing low-end mobile devices; zero licensing fees.
UX Complexity	✕ Cluttered, complex portals built for accountants.	✓ Minimalist, cashier-focused interface with automated barcode scanning.
Academic Constraints	✕ Relies on massive third-party libraries and cloud databases.	✓ Built entirely from scratch using raw JS and a local Oracle DB constraint.

Core Project Objectives

Zero Latency Downtime

Enable cashiers to record Sales (OUT) and Stock received (IN) instantly, regardless of network state.

Conflict-Free Auto Synchronization

Replay offline mutations idempotently and in strict FIFO (First-In-First-Out) order upon network recovery.

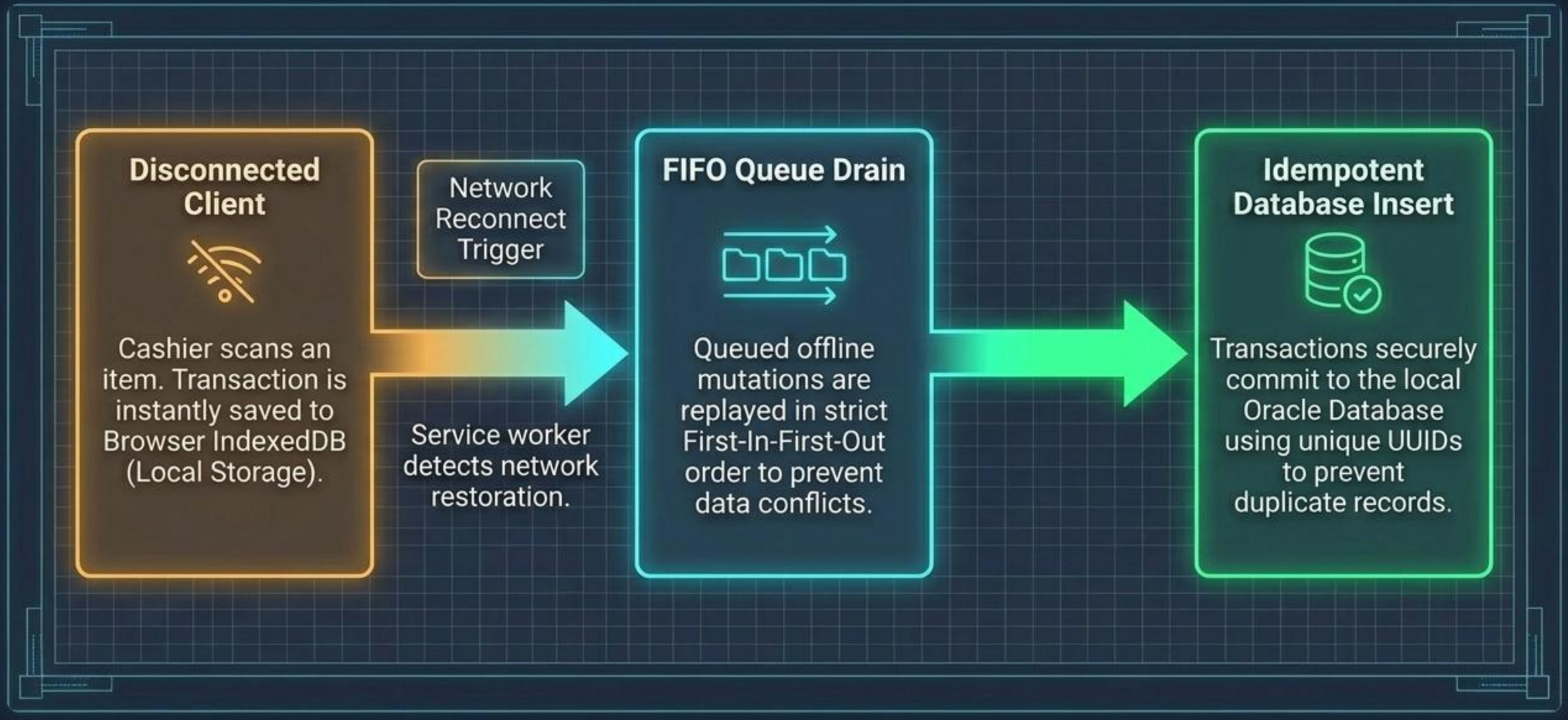
Native Hardware Integration

Access native camera modules for barcode scanning and trigger synthesized Web Audio scan beeps without external apps.

Resource Constraints Compliance

Deliver a production-grade system strictly adhering to university guidelines: Vanilla CSS, raw JavaScript, and Oracle DB.

The Offline-First Sync Engine



The “Zero-Framework” Tech Stack

Frontend Core

HTML5, CSS Variables, ES6 JavaScript Modules.

- Web MediaDevices (getUserMedia) & Web Audio API (Synthesizer).

PWA Engine & Backend

Service Worker (caching) + Browser IndexedDB.

- Node.js (ESM) Custom HTTP server: hand-built router, middleware, body parser, CORS handler (Zero Express).

Database & Infra

Oracle Database XE 21c (Local Machine).

- oracledb v6 Thin mode (No Instant Client). Docker Compose for production.

Role-Based Functional Boundaries

Cashier & Staff (Operational)

- Secure Login & Session validation
- Log transactions (Sale OUT / Stock Received IN)
- Dynamic autocomplete product search
- Native camera barcode autofill

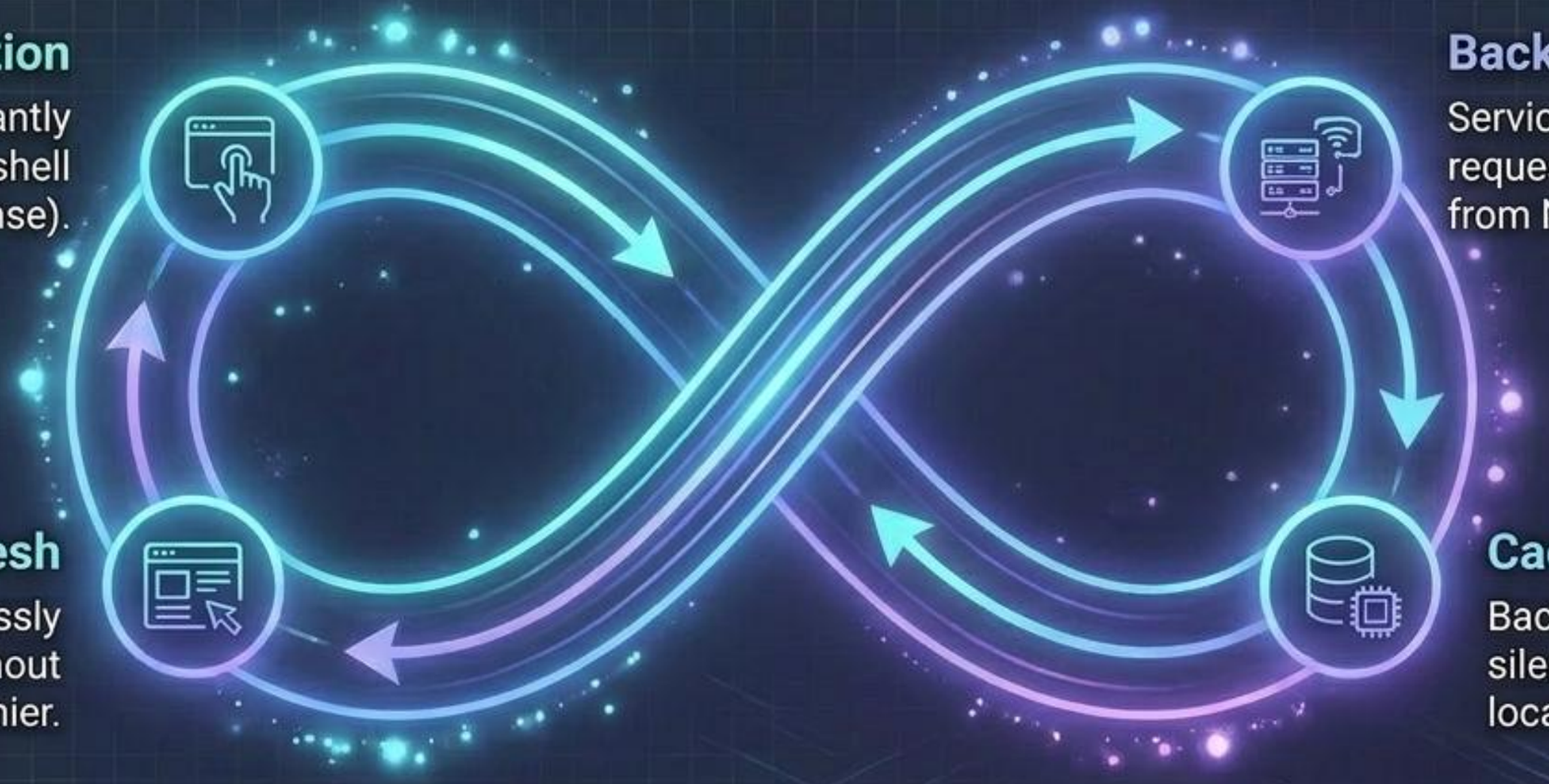
Administrator (Management)

- Full Product CRUD (Create, Edit, Delete)
- Set custom stock depletion alerts
- Staff Account Registry & active status toggling
- Dashboard Analytics: Live stock counts, canvas-drawn sales graphs, and expiry warnings

System Performance, Reliability, and Security Baselines

User Action

UI loads instantly from cached app shell (<50ms response).



Background Fetch

Service Worker silently requests latest data from Node.js server.

UI Refresh

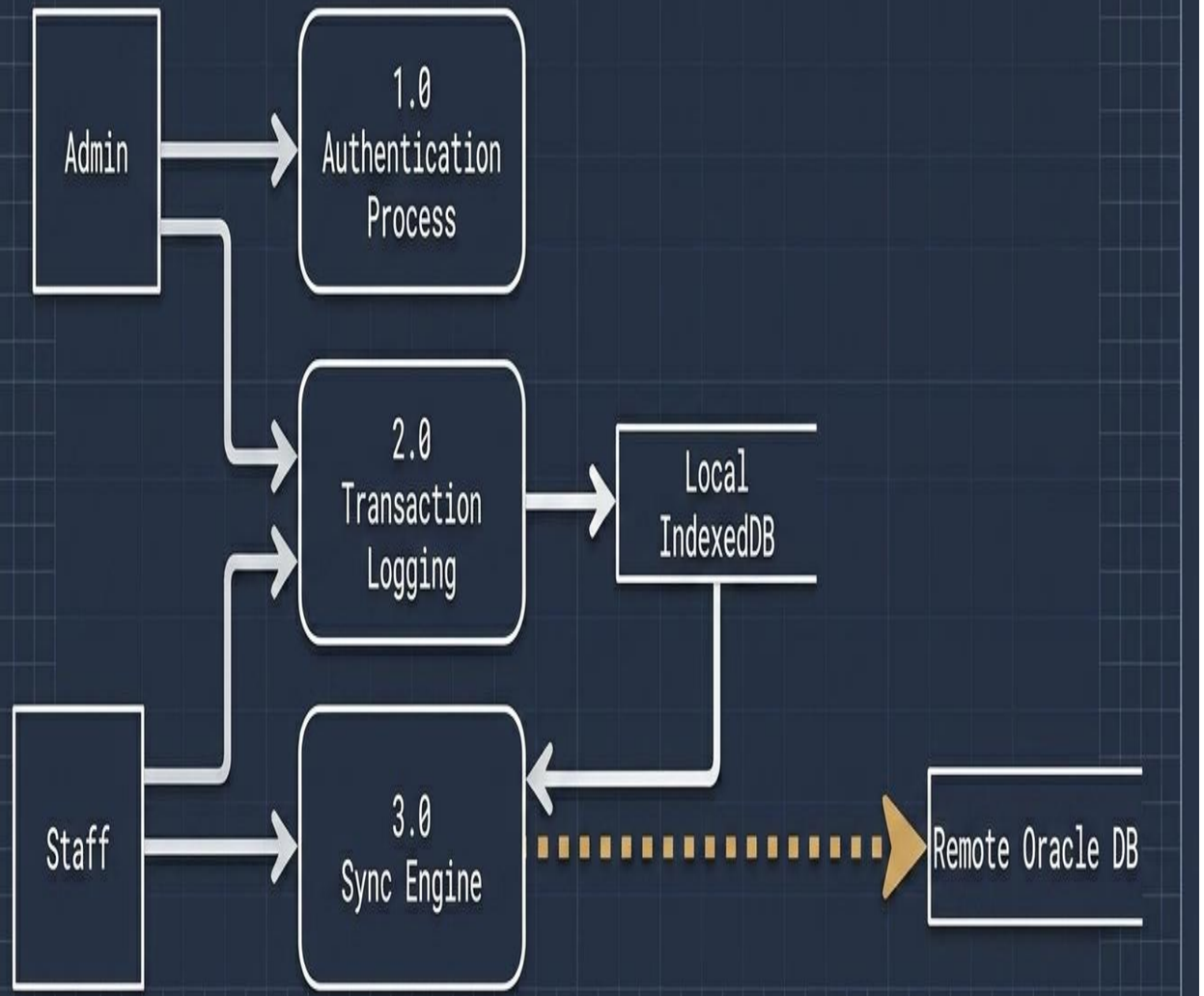
Interface seamlessly updates without blocking the cashier.

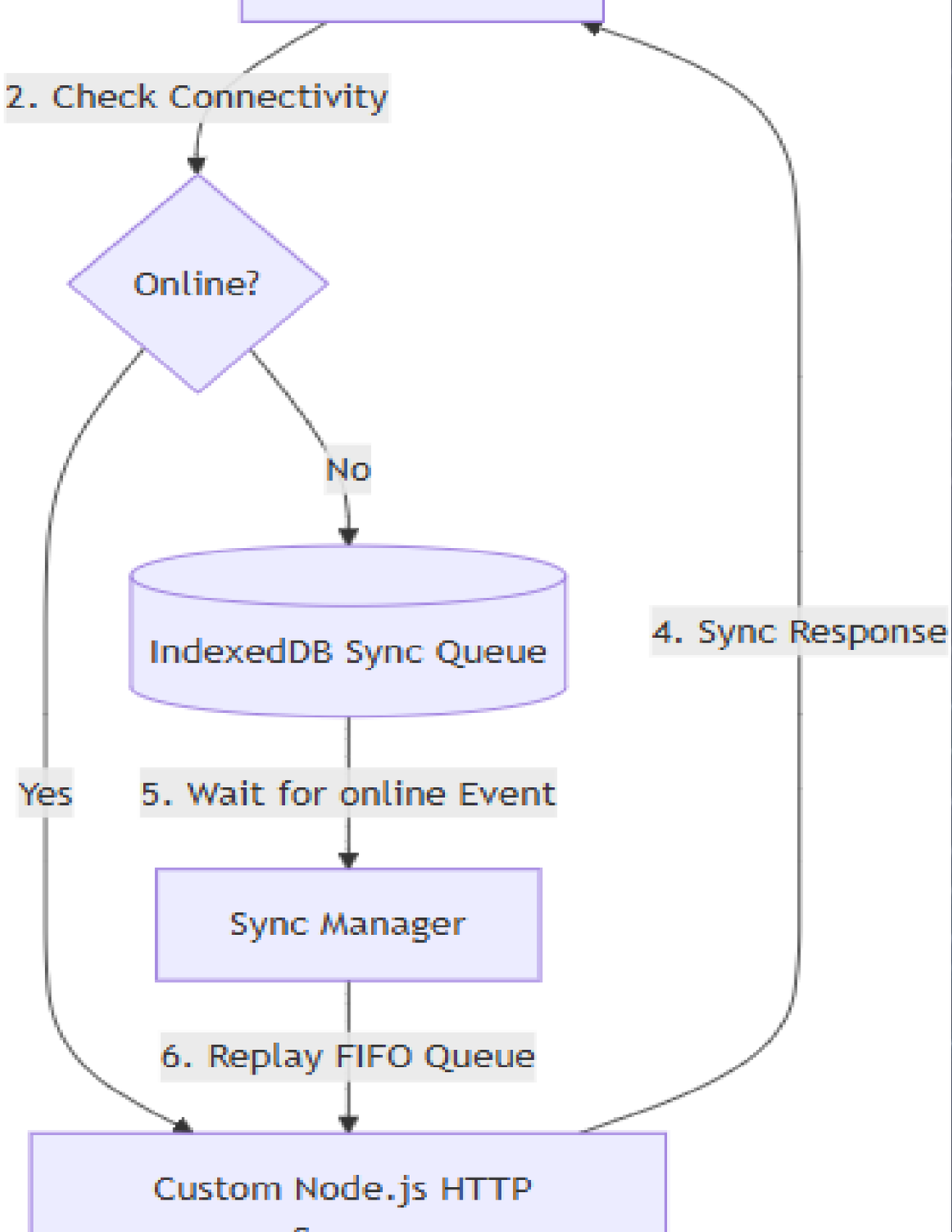
Cache Update

Background data silently updates the local cache.

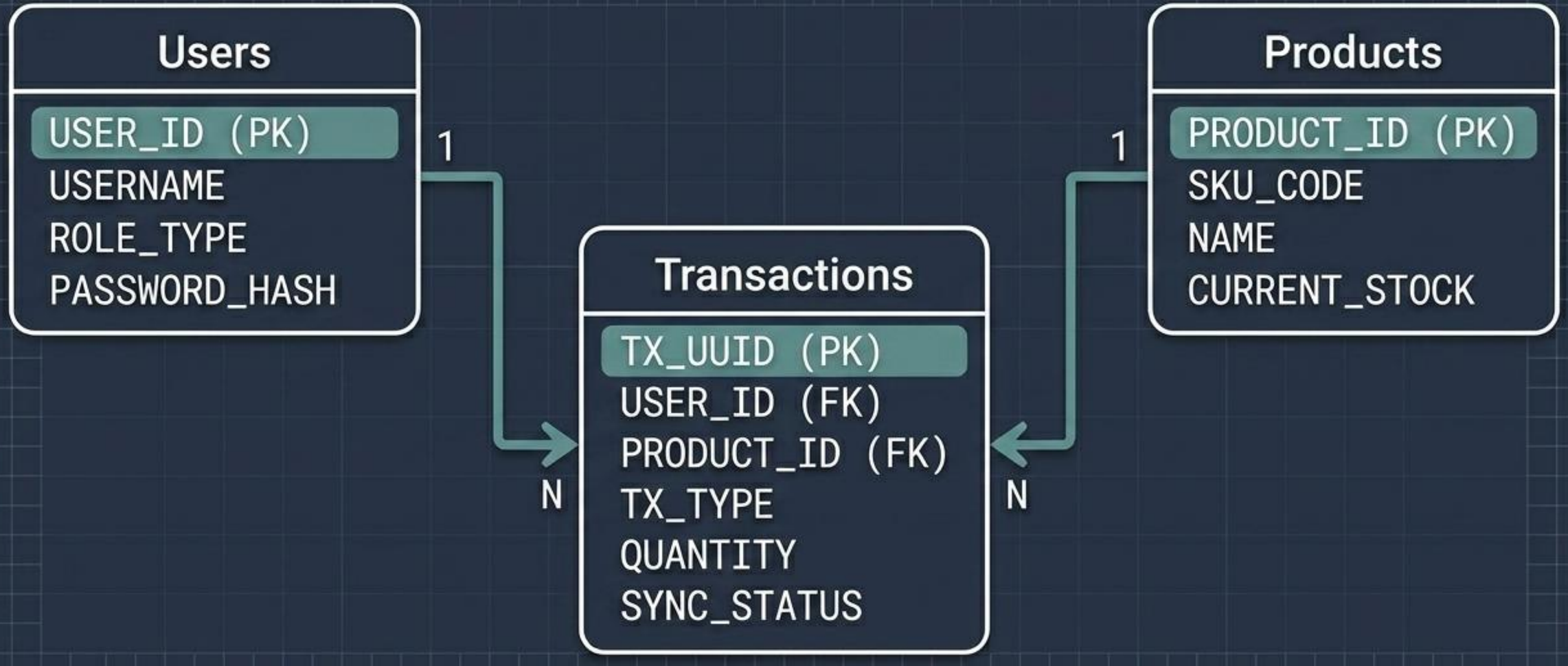
Reliability: Client UUID checks prevent duplicate inserts.
Security: Cryptographic Bcrypt hashing + JWT authentication.

Transaction Data Flow Across the Network Boundary





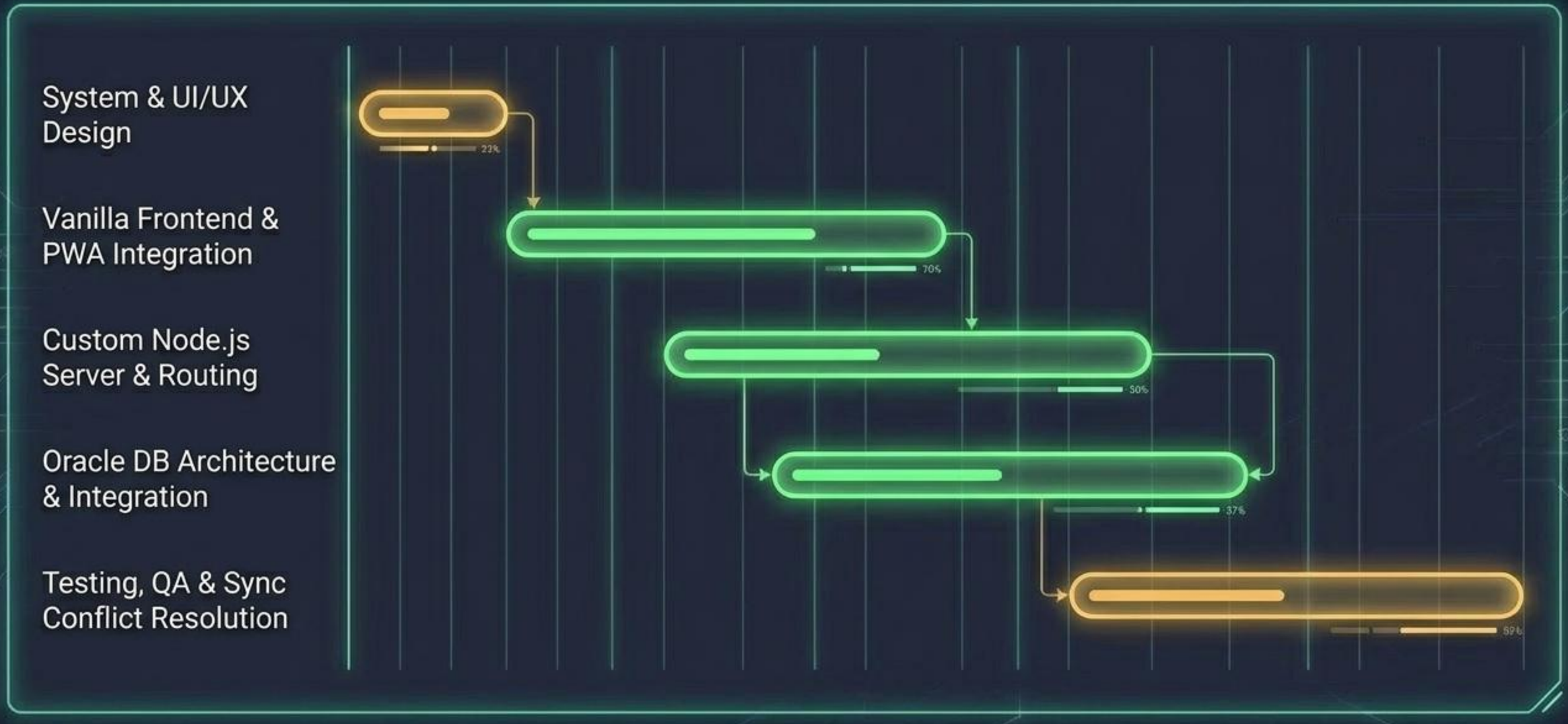
Relational Database Architecture



Executing the Schema in Oracle Database

```
CREATE TABLE transactions (
    tx_uuid VARCHAR2(36) PRIMARY KEY,
    user_id NUMBER NOT NULL,
    product_id NUMBER NOT NULL,
    tx_type VARCHAR2(10) CHECK (tx_type IN ('IN', 'OUT')),
    quantity NUMBER NOT NULL,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    sync_status VARCHAR2(15) DEFAULT 'PENDING',
    CONSTRAINT fk_user FOREIGN KEY (user_id) REFERENCES users(user_id),
    CONSTRAINT fk_product FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

Project Development Timeline



System Resilience (SWOT Analysis)

Strengths

- 100% offline uptime via IndexedDB/Service Workers;
- Ultralight load sizes (zero JS framework overhead);
- Conflict-free transactions via client UUIDs.

Weaknesses

- Database updates fundamentally rely on client synchronization triggers;
- Camera barcode detection depends on modern browser support.

Opportunities

- Expansion into multi-terminal retail point-of-sale setups;
- Integration with local hardware receipt printers.

Threats

- Device clear-cache commands could inadvertently wipe unsynced offline data;
- Evolving browser security standards for native camera feeds.

The Business Case: Cost vs. Value Delivered

Design & UI/UX:	**10k-15k BDT**
Frontend Dev:	**45k-60k BDT**
Backend Dev:	**60k-80k BDT**
Testing & QA:	**15k-25k BDT**
Domain/Hosting: 5k-10k BDT ~~(Bypassed via Local Dev)~~	
Oracle Database: 12k-24k BDT ~~(Bypassed via Oracle XE 21c Free Local)~~	

Validating the Power of Offline-First Web Technologies

- Bazar-Trace operates as a fully functional, offline-resilient inventory application tailored for low-connectivity retail.
- Strictly satisfies all academic requirements by exclusively utilizing pure vanilla code and relational Oracle schemas.
- Successfully demonstrates how modern native Web APIs can seamlessly replace heavy, expensive commercial frameworks.

Thank You / Q&A

Bazar-Trace Defense
University of Scholars | Student ID: 231010687
Supervisor: Reduanul Bari Subhon